

Scaling Certificate Issuance with EJBCA + Autoscaling Nitro Enclaves + KMS

A PKCS#11-fronted, load-balanced signing fabric — FIPS 140-3 Level 1 software custody

Purpose	Scale workload-identity cert issuance to any volume without an HSM cluster
Signing custody	FIPS 140-3 Level 1 — Go Cryptographic Module #5247 (GOFIPS140=v1.0.0), in a Nitro Enclave
Trust anchoring	One workload sub-ICA, signed by the existing root, sealed once, shared fleet-wide
EJBCA integration	Config-loaded PKCS#11 module (no EJBCA rebuild) → mTLS → enclave fleet
Scaling	EC2 Auto Scaling Group behind an NLB; scales on sign-rate; no session affinity
Document date	2026-07-09
Author	Kishore Mooli

1 Problem statement — why AWS CloudHSM does not fit high-scale workload identity

Workload identity at scale means issuing large volumes of short-TTL certificates, elastically, across regions, increasingly with post-quantum agility. Evaluated against that profile, an AWS CloudHSM cluster has six structural limitations. Each below is stated with the AWS-documented basis (see §10 references); where a common claim is an overstatement, it is corrected so the argument holds up to scrutiny.

1.1 Cost of an HA cluster is prohibitive at scale

CloudHSM is billed per HSM per hour, 24×7, for peak provisioned capacity — you pay whether the HSMs are saturated or idle. Per-HSM signing throughput is finite: measured ~1,600 ECDSA P-384 signatures/sec per hsm2m.medium node (P-256 ~2,500/node). At ~\$1.45/HSM-hour (~\$1,059/HSM-month, us-east-1), high availability needs at least two HSMs across two AZs — and because a CloudHSM cluster cannot span regions, seamless multi-region failover means a second full cluster. The table sizes each of two regions for the full P-384 load.

Signatures/sec (P-384)	HSMs / region (~1,600/HSM)	2-region total	\$ / month	\$ / year
2,000	2	4	\$4,236	~\$51k
3,000	2	4	\$4,236	~\$51k
5,000	4	8	\$8,472	~\$102k
8,000	5	10	\$10,590	~\$127k
10,000	7	14	\$14,826	~\$178k

Assumptions: measured throughput — ~1,600 ECDSA P-384 sig/sec per hsm2m.medium node (P-256 ~2,500/node); hsm2m.medium ~\$1.45/HSM-hr (~\$1,059/mo, us-east-1; ~10% higher in some regions); each region sized for the full load so failover is seamless, both billed 24×7 at peak regardless of utilization. The 2,000–3,000/sec rows are floored at 2 HSMs/region for AZ redundancy. N+1 (no throughput loss on a single HSM failure) adds one HSM per region. By contrast the EC2 signing fleet does ECDSA-P384 in software (thousands/sec per core) and scales with demand — a small on-demand fleet matches the HSM cluster at a fraction of the always-on cost, and KMS charges only the per-boot release.

1.2 No autoscaling — capacity must be provisioned for peak

AWS states plainly that CloudHSM clusters do not autoscale: to change capacity you add or remove HSMs yourself. There is no load-driven scaling policy, so you must provision for peak and carry that cost continuously. Adding/removing HSMs is an API call you must automate and sequence carefully, and each HSM has a hard ceiling of 2,048 open connections — a limit that, once hit under a load spike, causes application-level outages with no elastic relief valve.

1.3 Regional service — cross-region failover is manual

A CloudHSM cluster is a regional resource that cannot span regions, and AWS provides no native DR. Cross-region resilience is a manual clone: copy a cluster backup to the second region, create a cluster from it, then keep the two in sync by hand — creating users on both, syncing passwords, and syncing newly created keys. Failover is therefore an operator procedure, not a seamless regional cutover, so a regional impairment translates into a signing outage until the manual failover completes.

1.4 No production post-quantum

The algorithm set is bound to the HSM firmware — you cannot add new algorithms — and there is no production post-quantum signing (only experimental firmware PQC; ML-DSA is not GA/validated). For a PQC-ready workload PKI this is the real gap: the software enclave module can issue ML-DSA-65 and hybrid composite (ECDSA-P384 + ML-DSA-65) certificates today, which the HSM firmware cannot.

1.5 Signing is bound to firmware mechanisms — no hybrid/composite construction

Signing is limited to the firmware's mechanisms (CKM_ECDSA, CKM_RSA_PKCS/PSS). There is no room for custom signing logic or X.509 alt-signature / composite construction inside the HSM. A software signer can implement the draft-ietf-lamps-x509-alt composite and pure-PQ shapes the HSM cannot.

1.6 Metrics are infrastructure-level, with a hard connection cliff

The metrics are infrastructure signals — open connections, session/token key counts, HsmSessionCount, processor junction temperature — not signing throughput or latency. You cannot directly observe operations/sec or queue depth, which makes load-based capacity planning guesswork, and the 2,048-connection ceiling is an outage cliff you can hit before any obvious warning. Elastic, request-rate-driven autoscaling is impossible on signals like these.

1.7 Net

For high-scale workload identity — elastic demand, multi-region, PQC-ready, cost-sensitive — CloudHSM is cost-prohibitive at HA (§1.1), cannot autoscale (§1.2), needs manual cross-region DR (§1.3), is fixed to firmware algorithms with no production PQC (§1.4–1.5), and offers only infrastructure metrics with a hard connection cliff (§1.6). These make an HSM-backed workload-identity CA impractical and expensive at scale.

2 Proposed solution

Replace the HSM cluster with a PKCS#11-fronted, load-balanced fleet of AWS Nitro Enclaves that release one shared workload sub-ICA from AWS KMS under hardware attestation, and sign in-enclave. EJBCA reaches it through a config-loaded PKCS#11 module — no EJBCA rebuild — that forwards raw signatures over mTLS to the fleet.

- **Elastic + cost-scaling.** An EC2 Auto Scaling Group scales on sign-rate; you pay for the load you serve, not peak 24x7 HSM hours.
- **Multi-region-friendly.** The ICA is sealed once to inert ciphertext in SSM/Secrets Manager; any region's fleet releases the same key — no manual cluster cloning.
- **PQC-capable.** The software module issues ML-DSA-65 and hybrid composite certs the HSM firmware cannot.
- **App-level metrics.** Sign-rate, latency and NLB flow counts drive real autoscaling and capacity planning.
- **No EJBCA changes.** EJBCA loads a stock PKCS#11 module by config; the same signing fabric also serves cert-manager natively.

The one honest trade-off. Custody is FIPS 140-3 Level 1 (a validated software module run attested in a Nitro Enclave), not the Level 3 hardware custody CloudHSM provides. For workload-identity leaf/SVID

issuance — high volume, short TTL, elastic, PQC-ready — L1-attested-in-enclave is the appropriate and far more scalable point on the curve. Reserve L3 (KMS/CloudHSM) for the offline/near-offline root.

2.1 Cost at the same loads — head-to-head

Same ECDSA P-384 loads, same two-region base. The signing tier is 2× c5.4xlarge per region (4 instances) with autoscaling. A 16-vCPU c5.4xlarge running software ECDSA-P384 is expected to sustain well above 5,000 signs/sec (validate by load test), so a 2-instance/region baseline covers the 10,000/sec peak with headroom — the fleet stays flat while CloudHSM must add HSMs linearly.

Sig/sec (P-384)	CloudHSM (HSMs → \$/yr)	Our solution (c5.4xlarge → \$/yr, on-demand)
2,000	4 → ~\$51k	4 → ~\$24k
3,000	4 → ~\$51k	4 → ~\$24k
5,000	8 → ~\$102k	4 → ~\$24k
8,000	10 → ~\$127k	4 → ~\$24k
10,000	14 → ~\$178k	4 → ~\$24k

Our-solution assumptions: c5.4xlarge ~\$0.68/hr on-demand (~\$496/mo, us-east-1); 4 instances (2 per region) ≈ \$1,986/mo ≈ \$24k/yr. Nitro Enclaves add no charge (carved from the parent instance); NLB + per-boot KMS release + SSM add ~\$1k/yr. The line is flat because 2× c5.4xlarge/region covers the peak; autoscaling adds instances only above that (and scales down off-peak). Net: ~2× cheaper at the lowest load and ~7× cheaper at 10,000/sec — and the fleet tracks demand rather than always-on peak HSM hours.

3 Architecture

EJBCA (stock, HA) builds each certificate and calls a config-loaded PKCS#11 module for the signature; the module forwards raw-sign over mTLS to an NLB in front of an autoscaling enclave-signer fleet. Every enclave released the same workload ICA from the sealed blob in SSM, so the NLB needs no session affinity.

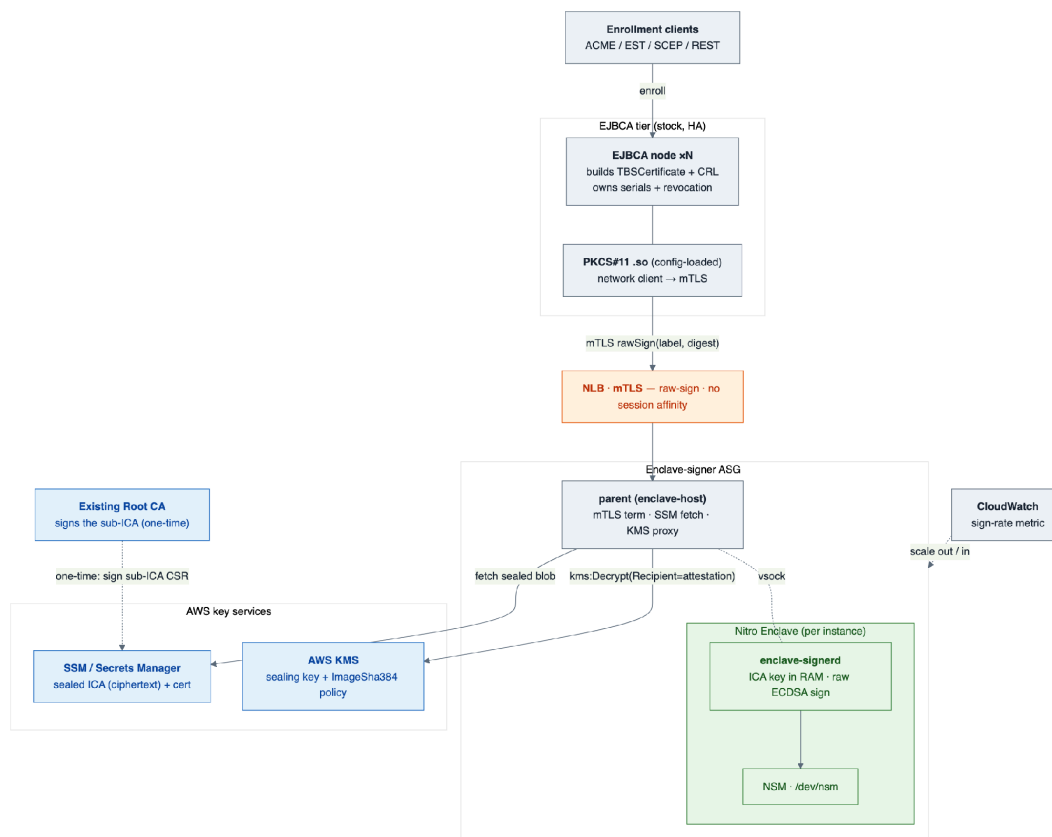


Figure 1 — EJBCA → config-loaded PKCS#11 module → mTLS NLB → autoscaling Nitro enclave-signer fleet, all releasing one shared ICA from SSM under the KMS attestation gate.

3.1 Components

Component	Role
EJBCA tier (stock, HA)	Builds TBSCertificate/CRL; owns serial space + revocation; loads the PKCS#11 module
PKCS#11 module (.so)	Config-loaded into EJBCA (no rebuild); a network client that forwards C_Sign over mTLS
NLB (mTLS)	Fronts the signer fleet; round-robins raw-sign; no affinity (replicated key)
Enclave-signer ASG	Autoscaling EC2 Nitro instances; scales on sign-rate; Spot-capable
parent (enclave-host)	mTLS termination · SSM fetch · KMS vsock proxy · vsock bridge (no key access)
Nitro Enclave (enclave-signerd)	Holds the released ICA in RAM; performs raw ECDSA/PQC signing; /dev/nsm attestation
SSM / Secrets Manager	Stores the sealed ICA (inert ciphertext) + intermediate cert

AWS KMS	Sealing key + attestation-gated release policy (kms:RecipientAttestation:ImageSha384)
Existing Root CA	Signs the workload sub-ICA once, anchoring it in the trust path

4 One-time ICA provisioning & KMS sealing

The workload sub-ICA is created and sealed exactly once — never per instance. Best practice is to generate and seal it inside a one-time bootstrap enclave so the private key is born and dies inside an enclave; only the ciphertext and cert leave.

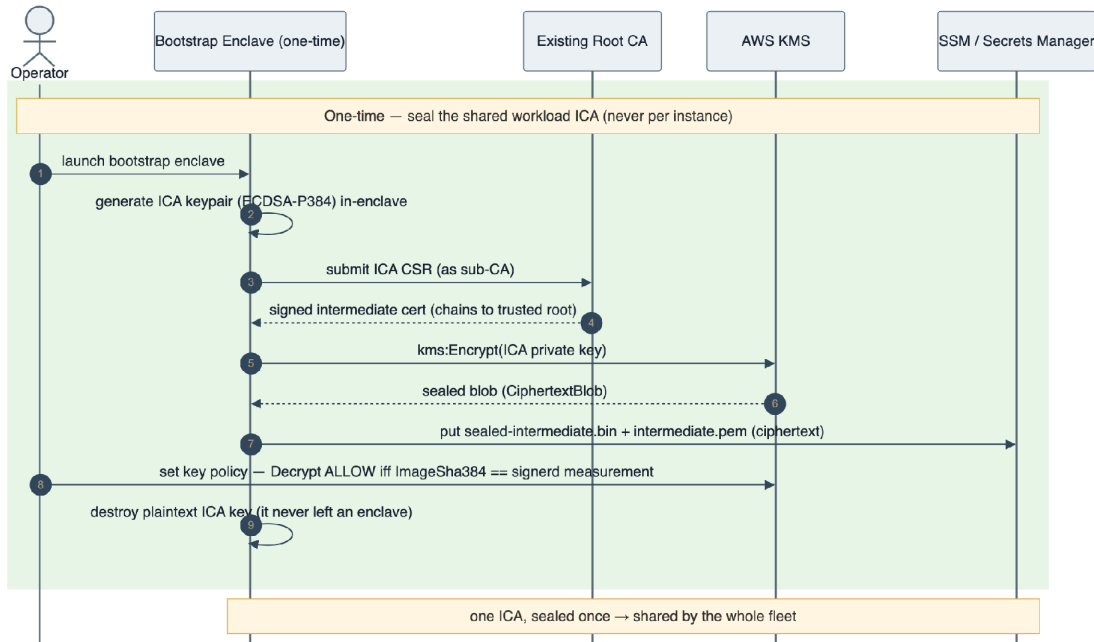


Figure 2 — One-time provisioning: generate ICA in-enclave, root signs the sub-ICA CSR, kms:Encrypt seals the key, ciphertext published to SSM, KMS policy bound to the enclave measurement.

4.1 Why one ICA, shared

Every enclave from the same image has the same measurement, so the same KMS policy releases the same key to all of them — one issuer identity across the whole fleet. Generating an ICA per instance would fork the issuer (N serial spaces, N revocation points) and defeat the model; sealing once and sharing the ciphertext is what makes autoscaling coherent.

5 Auto-scaling & attested key release

A new instance only serves traffic after it has attested and released the ICA — so the fleet grows and shrinks with load while every serving node provably holds the same key.

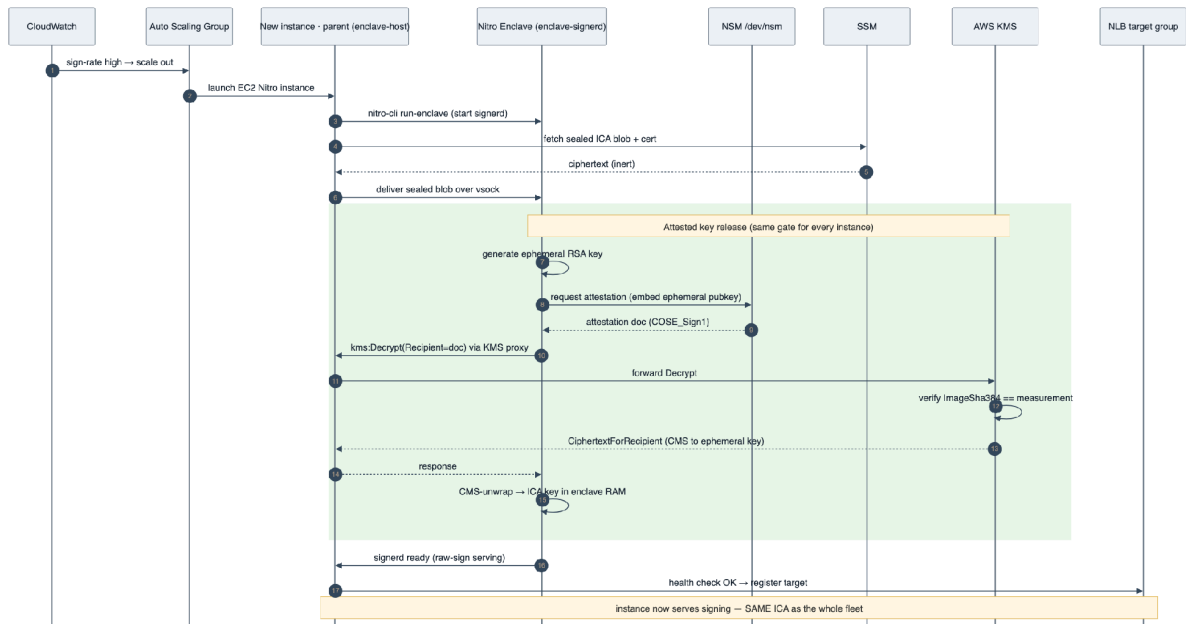


Figure 3 — Scale-out: ASG launches an instance; the enclave attests via NSM; KMS releases the ICA only if the measurement matches; the instance registers healthy and joins the NLB.

5.1 Scaling design

- **Separate ASG.** The enclave-signer fleet is its own Auto Scaling Group, independent of the EJBCA tier, so signing scales without scaling the JVM/DB tier.
- **Scale signal.** Target-tracking on raw-sign requests/sec per instance (or NLB active-flow count / CPU) — real application throughput, unlike CloudHSM's infra-only metrics.
- **Attested warm-up.** A new instance passes the NLB health check only after the attested release completes (a few seconds), so it never receives traffic without the key. Use a small warm pool / predictive scaling for spiky demand.
- **No affinity.** Replicated key $\Rightarrow$ any node signs identically; the NLB round-robins and scale-in just drains connections.
- **Multi-region.** Stand up the same ASG + sealed blob in each region; no cluster cloning or manual key sync — the ciphertext is identical and only attested enclaves can open it.

6 Certificate signing process

EJBCA assembles the certificate and asks the token only for the private-key signature over the digest — so the enclave exposes a raw-sign primitive, not full cert assembly.

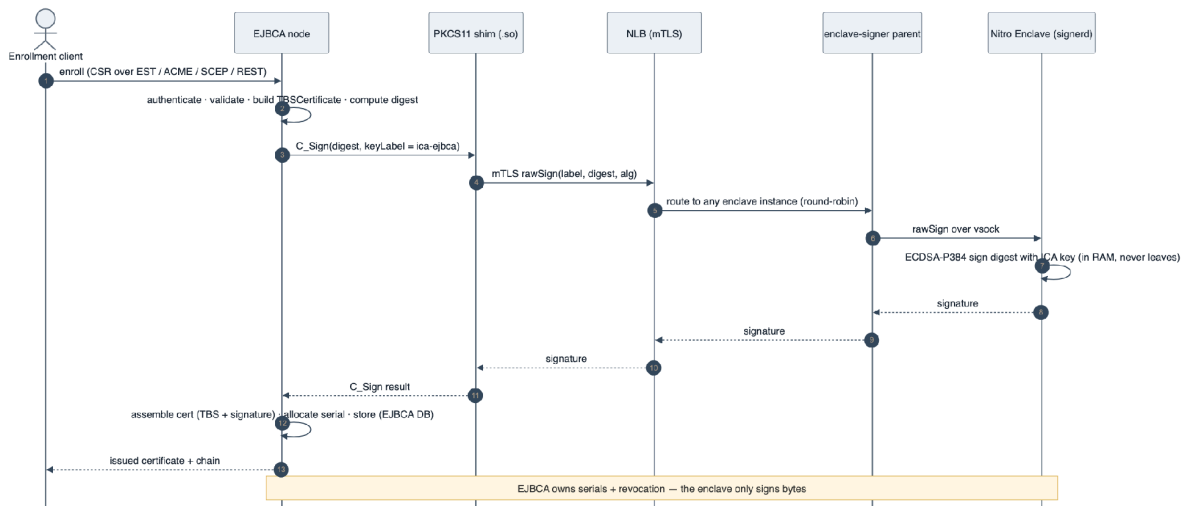


Figure 4 — Per-certificate signing: EJBCA `C_Sign` → PKCS#11 module → mTLS NLB → any enclave → raw ECDSA sign → back to EJBCA, which assembles and stores the cert.

6.1 What the enclave must expose

cert-manager uses full-assembly (CSR → finished cert); EJBCA needs raw-sign. Both live on the same daemon:

Consumer	Primitive
cert-manager	Full assembly — <code>SignRequest</code> → <code>SignResult</code> (exists today)
EJBCA via PKCS#11	<code>rawSign(keyLabel, digest, alg) → signature</code> , plus <code>getPublicKey / listByLabel</code> (to add)

EJBCA references the ICA as an existing key by label (e.g. `ica-ejbca`) with `C_GenerateKeyPair` disabled — the ICA is provisioned once, never per node. EJBCA owns the serial space and revocation; the enclave only signs bytes.

7 KMS operations reference

Operation	When	Who	Control
<code>kms:Encrypt</code> (seal)	Once, at provisioning	Bootstrap enclave	Restricted seal role; plaintext key never leaves an enclave
<code>kms:Decrypt</code> + Recipient (release)	Every instance boot	The attested enclave only	Granted ONLY by <code>kms:RecipientAttestation:ImageSha384 == measurement</code>
Key policy	Set once, updated on image change	KMS admin	Attestation-conditioned Decrypt; instance role has NO <code>kms:Decrypt</code>
Pinned key ID	Every release	Enclave config	Fixed key id blocks swapped-blob-under-other-key attacks

The instance IAM role may read the SSM ciphertext but must not hold `kms:Decrypt` — otherwise it bypasses attestation and the host could read the key. Decrypt authority comes solely from the attestation-conditioned key-policy statement.

8 Security controls

- **Attestation is the boundary.** The sealed blob is inert ciphertext; only an enclave whose `ImageSha384` matches the KMS policy can release the key. Distribute it freely (SSM, every host).
- **No IAM decrypt.** The instance role never has `kms:Decrypt` — the single most important control.

- **mTLS-lock the raw-sign endpoint.** It is a signing oracle: anyone who can call it can forge certs. Require mTLS client auth + private network + authz.
- **Verify key↔cert↔root at release.** Confirm the released key matches the pinned intermediate cert and chains to the expected root (defends against a swapped valid blob).
- **Least-privilege storage.** SSM write = provisioning role only; read = instance role only; versioned for rotation.
- **Key never in plaintext outside an enclave.** Generate+seal in a bootstrap enclave; the parent, SSM and anyone holding the blob only ever see ciphertext.

9 What to build

9.1 EJBCA side — config only, no rebuild

- Register the PKCS#11 module in conf/web.properties (library path + slot) and create a PKCS#11 crypto token in the Admin UI. Works on CE (SunPKCS11) and Enterprise (P11NG).
- The module is a network client (fork aws-kms-pkcs11): C_Sign forwards over mTLS to the NLB; implement the sign-only subset + find-by-label; disable key generation.

9.2 Enclave / fleet side

- Add rawSign(keyLabel, digest, alg) + getPublicKey + listByLabel to enclave-signerd (alongside the existing full-assembly path used by cert-manager).
- Add fetch-sealed-blob-at-boot in enclave-host (SSM/Secrets Manager → configured file) so ICA rotation needs no image rebuild.
- Add key↔cert↔root verification at release; swap the nitro runtime's LocalStack KMS client for the real default-chain client.
- Package the AMI/EIF, ASG, NLB, target-group health = attested-ready; target-tracking scale policy on sign-rate.

10 References

- AWS CloudHSM — Pricing — <https://aws.amazon.com/cloudhsm/pricing/>
- AWS CloudHSM — Performance — <https://docs.aws.amazon.com/cloudhsm/latest/userguide/performance.html>
- AWS CloudHSM — Scaling HSMs (add/remove; no autoscaling) — <https://docs.aws.amazon.com/cloudhsm/latest/userguide/add-remove-hsm.html>
- AWS CloudHSM — Resilience / DR (regional; manual) — <https://docs.aws.amazon.com/cloudhsm/latest/userguide/disaster-recovery-resiliency.html>
- AWS Security Blog — How to clone a CloudHSM cluster across regions — <https://aws.amazon.com/blogs/security/how-to-clone-an-aws-cloudhsm-cluster-across-regions/>
- AWS CloudHSM — PKCS#11 supported mechanisms — <https://docs.aws.amazon.com/cloudhsm/latest/userguide/pkcs11-mechanisms.html>
- AWS CloudHSM — CloudWatch metrics — <https://docs.aws.amazon.com/cloudhsm/latest/userguide/hsm-metrics-cw.html>
- AWS CloudHSM — Cluster management best practices (2,048 connections) — <https://docs.aws.amazon.com/cloudhsm/latest/userguide/bp-cluster-management.html>
- AWS KMS — Pricing — <https://aws.amazon.com/kms/pricing/>
- Amazon EC2 — c5.4xlarge pricing/specs — <https://instances.vantage.sh/aws/ec2/c5.4xlarge>
- Amazon EC2 — Spot Instances pricing — <https://aws.amazon.com/ec2/spot/pricing/>

- AWS Nitro Enclaves — Concepts (vsock, KMS attestation) — <https://docs.aws.amazon.com/enclaves/latest/user/nitro-enclave-concepts.html>
- EJBCA — Generic PKCS#11 Provider — <https://docs.keyfactor.com/ejbca/latest/generic-pkcs-11-provider>
- aws-kms-pkcs11 — PKCS#11 provider (fork base) — <https://github.com/JackOfMostTrades/aws-kms-pkcs11>